

# TASK 3: Expense Tracker with Analytics

Domain: FinTech | Level: Medium

---

## What is this project?

A command-line expense tracker that stores expenses in an SQLite database, supports adding, editing, deleting expenses by category, shows monthly totals, and alerts you when you exceed a budget.

## Key Concepts to Learn First

- JDBC (Java Database Connectivity) – connect Java to SQLite
- CRUD operations: CREATE, READ, UPDATE, DELETE
- SQLite database (single file, no server needed)
- Scanner for user input
- Simple analytics: grouping by month/category

## Step-by-Step Explanation

### Step 1 – Set up SQLite

Download `sqlite-jdbc-3.xx.jar` from GitHub (xerial/sqlite-jdbc). No installation needed.

### Step 2 – Create Database & Table

On first run, create an 'expenses' table with columns: id, date, category, amount, description.

### Step 3 – Add Expense

Use `PreparedStatement` to safely insert data (prevents SQL injection).

### Step 4 – View Expenses

`SELECT` all rows and print them in a table.

### Step 5 – Monthly Analytics

`GROUP BY strftime('%Y-%m', date)` to get totals per month.

### Step 6 – Budget Alert

After each insert, compare total for current month with budget limit.

## Complete Java Code

```
import java.sql.*;
import java.util.Scanner;

/**
 * Expense Tracker with SQLite
 * Dependency: sqlite-jdbc-3.xx.jar (from github.com/xerial/sqlite-jdbc)
 */
public class ExpenseTracker {

    // SQLite database file (created automatically)
```

```

static final String DB_URL = "jdbc:sqlite:expenses.db";
static final double BUDGET_LIMIT = 5000.0; // Monthly budget in your currency

public static void main(String[] args) throws SQLException {
    // ■■ Connect to SQLite database ■■
    Connection conn = DriverManager.getConnection(DB_URL);
    createTable(conn);

    Scanner sc = new Scanner(System.in);
    boolean running = true;

    while (running) {
        System.out.println("\n=== EXPENSE TRACKER ===");
        System.out.println("1. Add Expense");
        System.out.println("2. View All Expenses");
        System.out.println("3. Monthly Analytics");
        System.out.println("4. Delete Expense");
        System.out.println("5. Exit");
        System.out.print("Choose: ");
        int choice = sc.nextInt(); sc.nextLine();

        switch (choice) {
            case 1 -> addExpense(conn, sc);
            case 2 -> viewExpenses(conn);
            case 3 -> monthlyAnalytics(conn);
            case 4 -> deleteExpense(conn, sc);
            case 5 -> running = false;
            default -> System.out.println("Invalid choice.");
        }
    }
    conn.close();
    System.out.println("Goodbye!");
}

// ■■ Create table if it doesn't exist ■■
static void createTable(Connection conn) throws SQLException {
    String sql = ""
        CREATE TABLE IF NOT EXISTS expenses (
            id          INTEGER PRIMARY KEY AUTOINCREMENT,
            date         TEXT NOT NULL,
            category     TEXT NOT NULL,
            amount       REAL NOT NULL,
            description  TEXT
        )"";
    conn.createStatement().execute(sql);
}

// ■■ Add a new expense ■■
static void addExpense(Connection conn, Scanner sc) throws SQLException {
    System.out.print("Date (YYYY-MM-DD): "); String date = sc.nextLine();
    System.out.print("Category (Food/Travel/Bills/Other): "); String cat = sc.nextLine();
    System.out.print("Amount: "); double amount = sc.nextDouble(); sc.nextLine();
    System.out.print("Description: "); String desc = sc.nextLine();

    String sql = "INSERT INTO expenses(date,category,amount,description) VALUES(?,?,?,?)";
    try (PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, date);
        ps.setString(2, cat);
        ps.setDouble(3, amount);
    }
}

```

```

        ps.setString(4, desc);
        ps.executeUpdate();
    }
    System.out.println("Expense added!");

    // ■■ Budget Alert ■■
    String month = date.substring(0, 7); // "YYYY-MM"
    double total = getMonthTotal(conn, month);
    if (total > BUDGET_LIMIT) {
        System.out.printf("ALERT: You exceeded budget! Spent: %.2f / %.2f%n",
            total, BUDGET_LIMIT);
    }
}

// ■■ View all expenses ■■
static void viewExpenses(Connection conn) throws SQLException {
    System.out.printf("\n%-4s %-12s %-10s %-8s %-20s%n",
        "ID", "Date", "Category", "Amount", "Description");
    System.out.println("-".repeat(58));
    ResultSet rs = conn.createStatement()
        .executeQuery("SELECT * FROM expenses ORDER BY date DESC");
    while (rs.next()) {
        System.out.printf("%-4d %-12s %-10s %-8.2f %-20s%n",
            rs.getInt("id"), rs.getString("date"),
            rs.getString("category"), rs.getDouble("amount"),
            rs.getString("description"));
    }
}

// ■■ Monthly Analytics ■■
static void monthlyAnalytics(Connection conn) throws SQLException {
    System.out.println("\n=== MONTHLY TOTALS ===");
    String sql = ""
        SELECT strftime('%Y-%m', date) as month,
           category,
           SUM(amount) as total
        FROM expenses
        GROUP BY month, category
        ORDER BY month DESC"";
    ResultSet rs = conn.createStatement().executeQuery(sql);
    System.out.printf("%-10s %-12s %-10s%n", "Month", "Category", "Total");
    System.out.println("-".repeat(35));
    while (rs.next()) {
        System.out.printf("%-10s %-12s %.2f%n",
            rs.getString("month"),
            rs.getString("category"),
            rs.getDouble("total"));
    }
}

// ■■ Delete an expense ■■
static void deleteExpense(Connection conn, Scanner sc) throws SQLException {
    viewExpenses(conn);
    System.out.print("Enter ID to delete: ");
    int id = sc.nextInt(); sc.nextLine();
    conn.createStatement().executeUpdate("DELETE FROM expenses WHERE id=" + id);
    System.out.println("Deleted!");
}

// Helper: get total for a given month

```

```

        static double getMonthTotal(Connection conn, String month) throws SQLException {
            String sql = "SELECT SUM(amount) FROM expenses WHERE date LIKE ?";
            try (PreparedStatement ps = conn.prepareStatement(sql)) {
                ps.setString(1, month + "%");
                ResultSet rs = ps.executeQuery();
                return rs.next() ? rs.getDouble(1) : 0;
            }
        }
    }
}

```

## How to Compile & Run

```

# 1. Download: sqlite-jdbc-3.45.1.0.jar
#    from: https://github.com/xerial/sqlite-jdbc/releases

# 2. Compile:
javac -cp ".;sqlite-jdbc-3.45.1.0.jar" ExpenseTracker.java

# 3. Run:
java -cp ".;sqlite-jdbc-3.45.1.0.jar" ExpenseTracker

# The file expenses.db will be created automatically.

```

TIP: You'll learn JDBC • SQL CRUD • PreparedStatement • GROUP BY analytics